

Design Patterns

This report evaluates the application of software design patterns in the KIVI game project, focusing on GRASP and classic design patterns to enhance modularity and scalability.

Applied Patterns:

1. GRASP Patterns

- Information Expert: GameManager centralizes game state (e.g., saving/loading), ensuring cohesive data handling.
- Creator: GameManager instantiates Player/AIPlayer objects, adhering to object-creation responsibilities.
- Controller: GameManager acts as a facade, coordinating game flow (turns, dice, UI updates).
- Polymorphism: AIPlayer overrides takeTurn() for difficulty-based behaviors (Hard rerolls strategically).
- Singleton: GameManager.getInstance() ensures a single instance for consistent state management.

2. Protected Variations & Indirection

- Encapsulation in Player subclasses allows future AI extensions without modifying core logic. GameManager mediates UI-domain interactions, reducing direct dependencies.

Potential Improvements:

- Observer Pattern: Decouple UI (Grid, DiceRollGUI) from GameManager by implementing observers for real-time state updates (e.g., dice rolls, score changes).
- Factory Method: Introduce AIFactory to generate EasyAIPlayer/HardAIPlayer, isolating creation logic and supporting new AI types.
- Command Pattern: Encapsulate actions (e.g., "Save Game") into Command objects for undo/redo functionality and modular input handling.

Conclusion:

The project effectively utilizes GRASP principles (Controller, Polymorphism) and Singleton for state management. Future work should adopt Observer and Factory patterns to reduce coupling and enhance extensibility, aligning with best practices in object-oriented design.